



**Master Informatique, Gouvernance et
Transformation Digitale (iGov)**

Software engineering

**Design and Development of a
Parking Reservation Management
Application**

Réalisé par : Hajar CHAHBI & Manal Zerrouki

Encadré par : Abdelmoujoud ASSABIR

Année universitaire : 2024-2025

Plan :

Introduction.....	4
1. Contexte :	4
2. Problematic :	4
3. Solution :	4
Chapter 1: Needs Analysis and Feasibility.....	5
1. Project Objectives :	5
2. Functional and Non-functional Specifications:.....	5
2.1. Functional specifications :	5
2.2. Non-functional specifications :	5
• Security.....	5
• Availability.....	5
• Scalability.....	5
• Usability.....	6
• Maintainability.....	6
• Constraints.....	6
3. Project Timeline :	6
4. Deliverables :	7
5. Validation Criteria :	7
Chapter 2 : Conception.....	7
1. Modeling language :	7
1.1. Language UML :	7
1.2. Enterprise Architect :	7
1.3. Use Case.....	8
1.3.1. What is a Class Diagram?.....	8
1.3.2. Use Case Diagram.....	8
1.3.3. Description.....	9
1.4. Classe.....	9
1.4.1. What is a Class Diagram?.....	9
1.4.2. Class Diagram.....	10
1.4.3. Description.....	10
2. Logical Data Modeling.....	11
2.1. Purpose of the LDM :	11
2.2. LDM :	11
2.3. Description.....	11
2.3.1. Table: users.....	11
2.3.2. Table: parkings.....	12
2.3.3. Table: reservations.....	12
2.3.4. Relationships.....	12
Chapter 3 : implementation.....	12
1. Tools used.....	12
1.1. Backend.....	12

1.1.1. Python.....	12
1.1.2. SQLite.....	13
1.2. Front-end.....	13
1.3. ReportLab.....	14
2. Interfaces :.....	15
2.1. Page: Login.....	15
2.1.1. Purpose:.....	15
2.1.2. Layout and Features:.....	15
2.2. Page: Reserve a Parking Spot.....	16
2.2.1. Purpose:.....	16
2.2.2. Layout and Features:.....	16
2.3. Page: My Reservations.....	18
2.3.1. Purpose:.....	18
2.3.2. Layout and Features:.....	18
3. Test :.....	19
3.1. Unit testing.....	19
3.2. Integration test.....	19
3.3. Validation test.....	20
Conclusion générale.....	20
Bibliographie.....	20

Introduction

1. Contexte :

In urban cities, traffic congestion, lack of parking space, and poor utilisation of resources are continuously raising problems. A well-designed parking system can manage reservation, allotment, and parking space management quite efficiently.

2. Problematic :

Transportation is a key factor in perceived livability of a city, and for those relying on cars, available parking is crucial. This is true for all the vehicles alike : permanent residents, visitors, and drivers of delivery.

Poorly managed parking causes stress and inefficiency, with a broader impact on general traffic congestion due to the volume of vehicles driving slowly through streets searching for a vacant space. Ultimately, this may mean that people park illegally.

This is not just a nuisance: in many instances, it may be life-threatening when emergency service vehicles are blocked by such vehicles. Then, there are the environmental negative impacts: slow-moving cars moving through streets and delivery drivers keeping their engines running while parked on the sides-illegally-add to the woes of pollution, air quality, and noise levels.

3. Solution :

Our parking reservation management system is robust, innovative, featuring real-time tracking of availability, and user-centric design. The system is designed to offer maximum user convenience, optimum utilization of parking space, and thereby enhance the efficiency of operations by the administrators. The architecture is modular and scalable, expanding with future needs.

Chapter 1: Needs Analysis and Feasibility

1. Project Objectives :

- Simplify parking space management for users.
- Allow users to view available parking spaces.
- Provide a solution for booking and canceling reservations.
- Generate tickets with reservation details.

2. Functional and Non-functional Specifications:

2.1. Functional specifications :

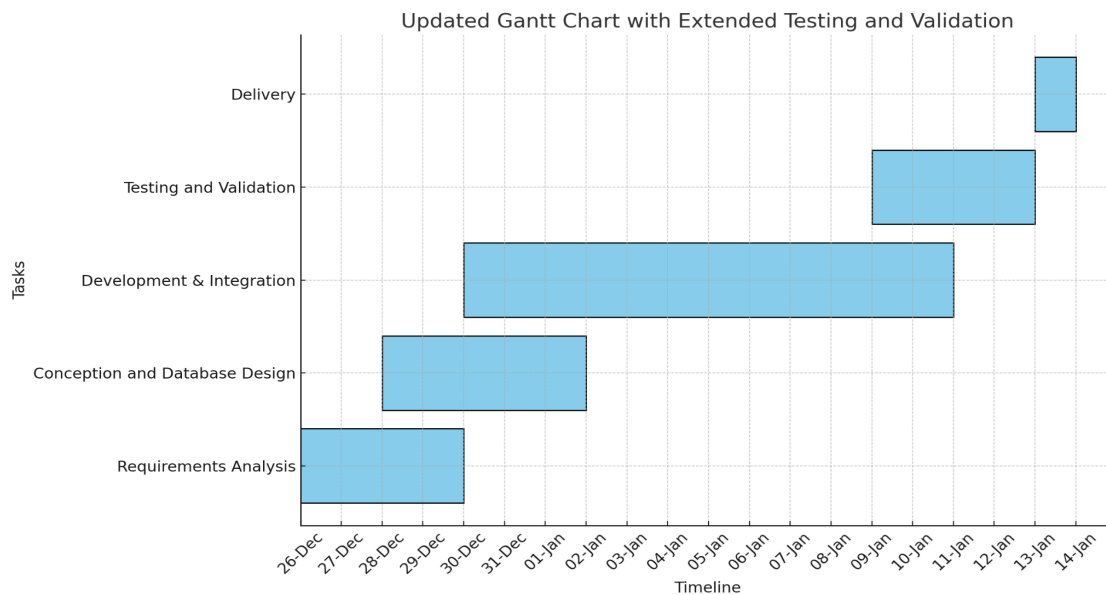
- The application should allow users to register by providing a unique username and password.
- The application should allow users to log in by authenticating their credentials (username and password) and granting access upon successful verification.
- The application should initialize parking slots by setting up predefined slots categorized as VIP, Handicap, or Standard, and assigning them to specific zones.
- The application should enable users to view all available parking slots, displaying details such as type, zone, and occupancy status.
- The application should automatically update and free up parking slots where reservations have expired.
- The application should allow users to view their existing reservations, including details like vehicle identifier, zone, start and end times, and assigned parking slots.
- The application should enable users to reserve a parking slot by providing their vehicle identifier, desired zone, and reservation period. The system should assign an available slot and confirm the reservation.
- The application should generate a PDF receipt for each reservation, detailing the reservation ID, vehicle identifier, reservation times, and assigned parking slot.

2.2. Non-functional specifications :

- **Performance**
 - The system should handle up to 100 concurrent users without significant performance degradation.
 - Queries to check availability or manage reservations must complete in under 2 seconds.
- **Security**
 - Passwords securely stored using hashing .
 - User sessions should be protected .
 - Sensitive actions need to require user authentication.
- **Availability**
 - The application should operate with a database uptime of at least 99.9%.
 - Updates to parking slots should occur automatically without user intervention.
- **Scalability**
 - The database schema supports expanding the number of parking zones and slots.

- Modular design should allow adding new features .
- **Usability**
 - Simple and intuitive user interface:
 - Login and reservation flows are accessible to non-technical users.
 - Receipts are generated in standard PDF format for compatibility.
- **Maintainability**
 - The codebase uses clear modularization.
 - Tables and data structures can be updated without breaking existing functionality.
- **Constraints**
 - A development time that is limited : 26 days.
 - Just 2 persons in the development team, which include the development of frontend, backend, and database.

3. Project Timeline :



Task	Start date	Duration (Days)	End Date	Description
Requirements Analysis	2024-12-26	4	2024-12-30	Analyze and document project requirements.
Conception and Database Design	2024-12-28	5	2025-01-02	Design system architecture and database schema.
Development & Integration	2024-12-30	12	2025-01-11	Develop, integrate components, and ensure functionality.
Testing and Validation	2025-01-09	4	2025-01-13	Test system performance and validate requirements.
Delivery	2025-01-13	1	2025-01-13	Deliver the completed project.

4. Deliverables :

Fully functional web applications.

Test reports demonstrating system performance and reliability.

5. Validation Criteria :

Successful deployment and testing phases.

Client feedback on system usability and performance.

Meeting predefined performance benchmarks.

Chapter 2 : Conception

1. Modeling language :

1.1. Language UML :

Unified Modeling Language, or UML, is a standardized modeling language that comprises an integrated set of diagrams, developed to help system and software developers describe, visualize, construct, and document the artifacts of software systems, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. UML is a very important part in object-oriented software development and in the process of software development. The UML basically expresses design using mainly graphical notation in software projects. The use of UML helps project teams communicate, explore potential designs, and validate the architectural design of the software.

1.2. Enterprise Architect :

We used Enterprise Architect to create Use case and Class diagrams. It is a sophisticated and user-friendly modeling and design platform tailored for modern needs, seamlessly blending exploration and innovation. Grasp the present, envision the future, and construct a dynamic model that is both robust and far-reaching. Your model will serve as a compass, empowering your vision, your team's goals, and your company's trajectory.



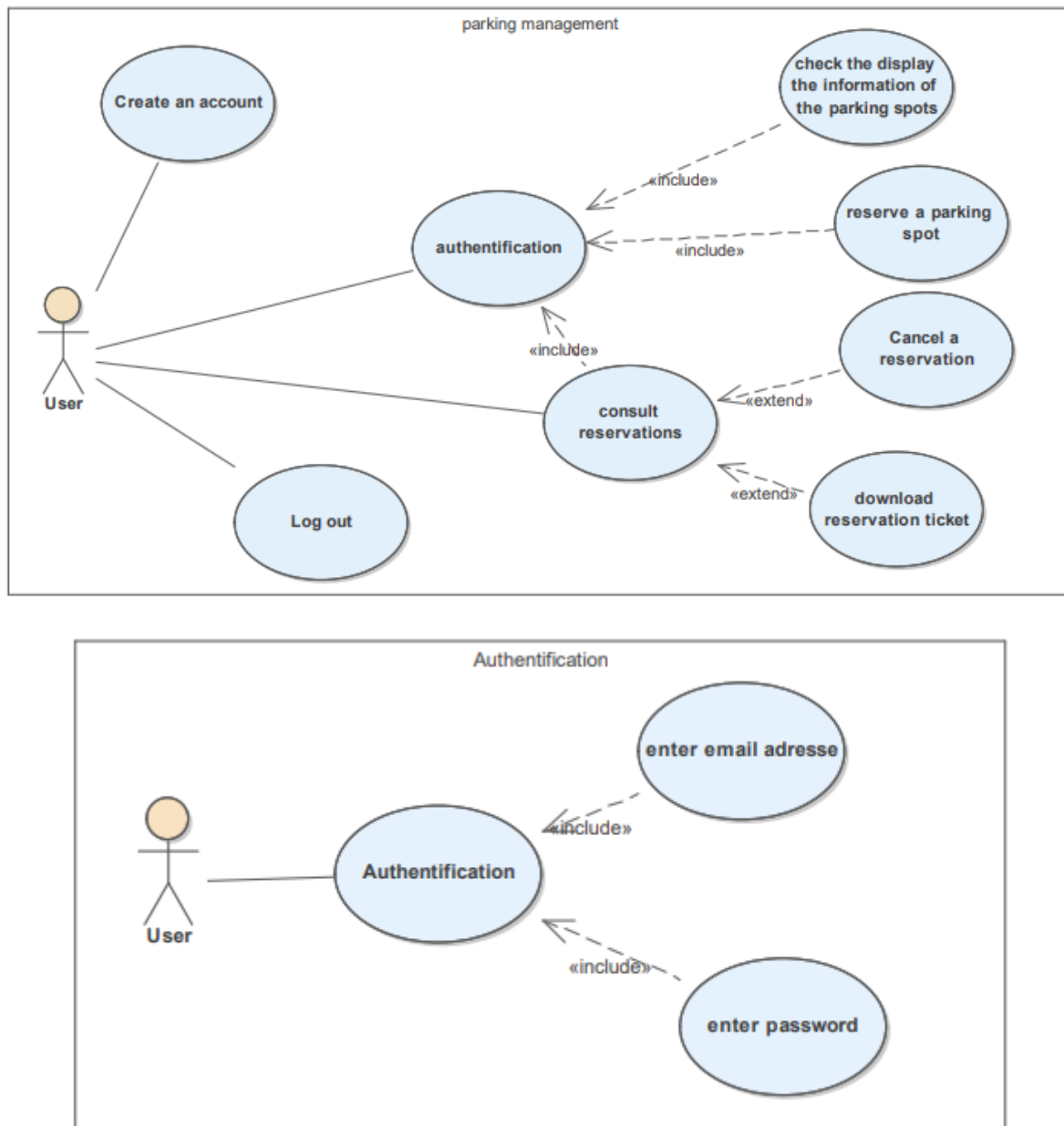
1.3. Use Case

1.3.1. What is a Class Diagram?

A use-case model describes the functional requirements of a system in terms of use cases. It is a model of the system's intended functionality, or use cases, and its environment, or actors. Use cases let you relate what you need from a system to how the system delivers on those needs.

As the use-case model is a very powerful planning instrument, it is generally used in all phases of the development cycle by all team members.

1.3.2. Use Case Diagram



1.3.3. Description

The first use case diagram represents the main functionalities of a user using the parking management system. It describes the core actions a user can perform, such as creating an account, logging out, and authentication. Authentication is the centerpiece in terms of subsequent access to other functionalities like consulting reservations and reserving parking spots. Specific features included here are to check the display and information of parking spots, to reserve a parking spot, to cancel the reservation, and to download the reservation ticket. The relationships are modeled by using <<include>> and <<extend>>, where shared functionalities such as authentication are prerequisites for other actions, and the extended functionalities such as downloading a reservation ticket is an added option within related processes.

The second diagram shows an expanded view of the authentication part of the system. The inputs required from the user are a key email address and a key password. In addition, each of these activities is related to the main authentication activity using <<include>> relationships, indicating that both these steps are necessary in order to satisfy the authentication process. It emphasizes the sequence of steps one has to go through when logging into a system, basically to access other features of the system.

1.4. Classe

1.4.1. What is a Class Diagram?

The class diagram is a central modeling technique that runs through nearly all object-oriented methods. This diagram describes the types of objects in the system and various kinds of static relationships which exist between them.

1.4.2. Class Diagram

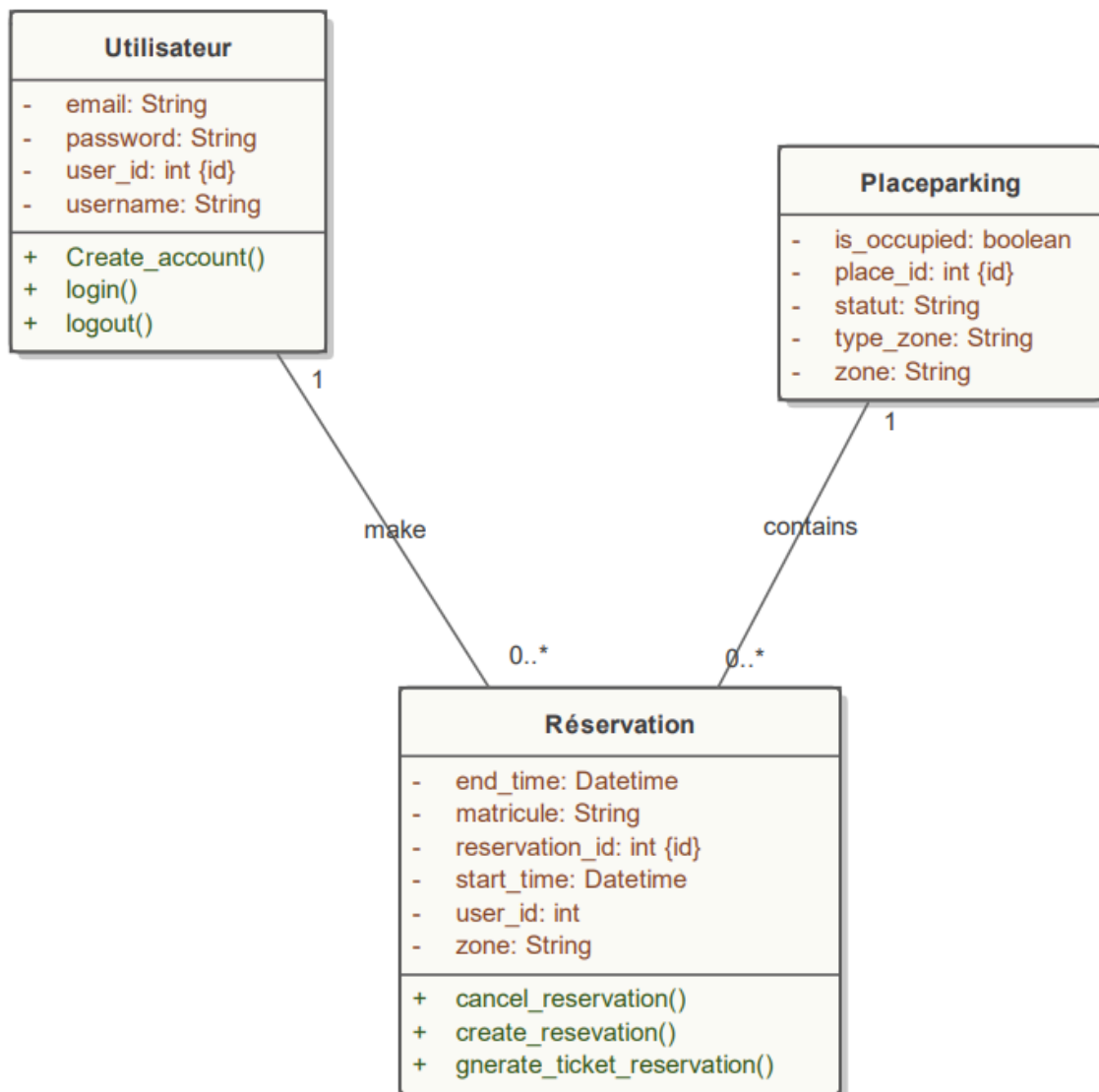


Figure 2.1:Diagramme de cas d'utilisation

1.4.3. Description

The class diagram includes the parking reservation system, which contains three major classes: Utilisateur, Parking, and Réservation.

- In the **Utilisateur** class, the user's information : e-mail, password, username, and user ID are available, and methods related to the creation of an account, log-in, and log-out are provided.
- The **Placeparking** class represents parking spots, with attributes indicating their occupancy status, unique ID, zone type, and location, but no defined methods.
- The **Réservation** class is responsible for parking reservations, and it has attributes such as start and end times, vehicle license plate number, and links to the user and parking spot. It also has methods for creating, canceling, and generating reservation tickets.

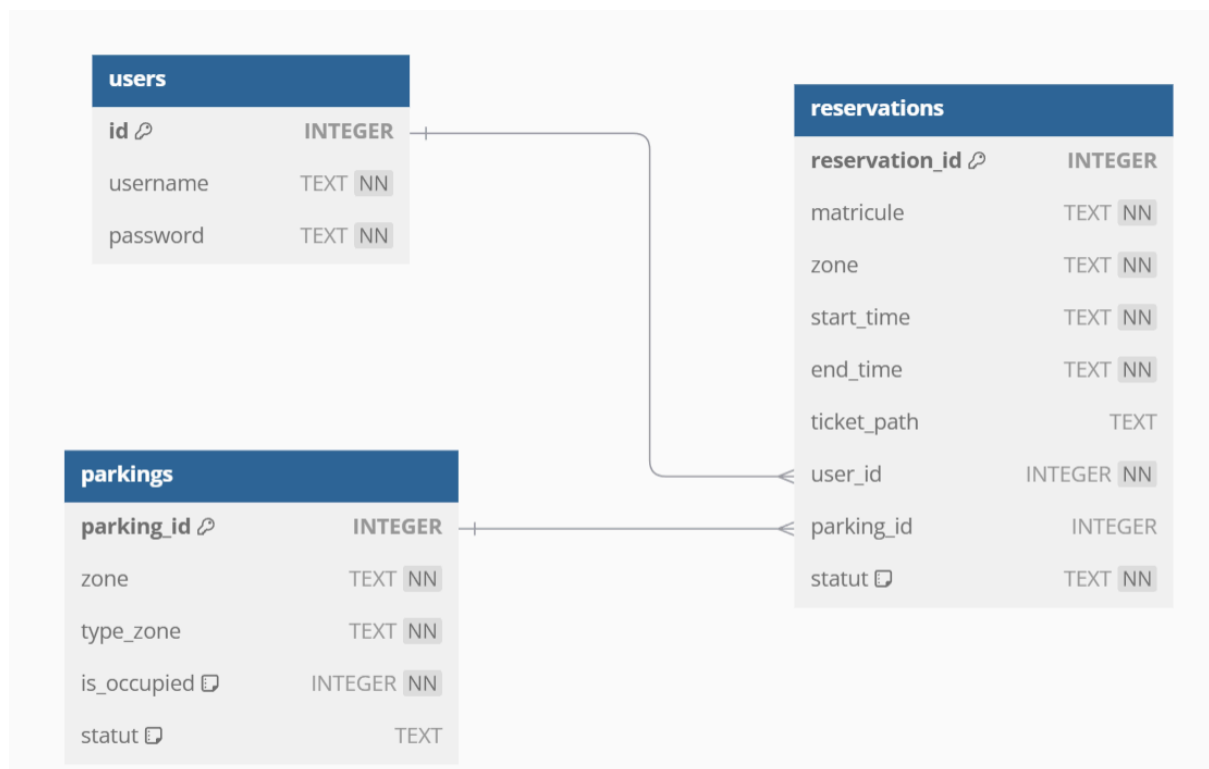
The relationships between classes are: a user can make multiple reservations (1 to 0.* relationship), and each reservation depends on a specific parking spot (1 to 1 relationship). It allows for the smooth management of parking spots and reservations, and it keeps the users responsible.

2. Logical Data Modeling

2.1. Purpose of the LDM :

The LDM is a type of data model utilized in the elaboration of data elements. It will provide the visual understandings of data entities, attributes, keys, and relationships to develop the visual representation of the data model. Such a model is uniquely independent from a specific database in order to set a basic structure for components of the semantic layer in data management systems. An LDM can be thought of as a blueprint in that it contains the definitions and characteristics of data elements that remain constant throughout technological changes.

2.2. LDM :



2.3. Description

2.3.1. Table: users

This table stores information about the users of the system. It includes:

- **id**: A unique identifier for each user (Primary Key).
- **username**: The username of the user, which must be unique and cannot be null.
- **password**: The password of the user, which is required and cannot be null.

2.3.2. Table: parkings

This table stores information about parking spots. It includes:

- **parking_id**: A unique identifier for each parking spot (Primary Key).
- **zone**: The zone or area where the parking spot is located (cannot be null).
- **type_zone**: The type of zone (e.g., residential, commercial) where the parking spot is located (cannot be null).
- **is_occupied**: Indicates whether the parking spot is occupied (default value is 0 for not occupied, and it cannot be null).
- **statut**: The status of the parking spot (default value is "Disponible" for available).

2.3.3. Table: reservations

This table stores information about parking reservations made by users. It includes:

- **reservation_id**: A unique identifier for each reservation (Primary Key).

- **matricule**: The vehicle's license plate number, which must be unique and cannot be null.
- **zone**: The zone or area of the parking reservation (cannot be null).
- **start_time**: The start time of the reservation (cannot be null).
- **end_time**: The end time of the reservation (cannot be null).
- **ticket_path**: A path or reference to the reservation ticket (optional).
- **user_id**: A foreign key linking the reservation to a specific user in the users table (cannot be null).
- **parking_id**: A foreign key linking the reservation to a specific parking spot in the parkings table (optional).
- **statut**: The status of the reservation (default value is "Disponible" for available, and it cannot be null).

2.3.4. Relationships

1. **reservations.user_id** → **users.id**: Links each reservation to the user who made it.
2. **reservations.parking_id** → **parkings.parking_id**: Links each reservation to the parking spot being reserved.

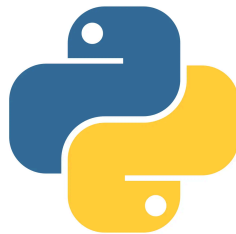
Chapter 3 : implementation

1. Tools used

1.1. Backend

1.1.1. Python

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Its high-level inbuilt data structures, combined with dynamic typing and binding, make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Python supports modules and packages. This encourages a large degree of program modularity and code reuse. The Python interpreter and the extensive standard library are freely available in source or binary form for all major platforms and can be freely distributed.



1.1.2. SQLite

SQLite is a C-language library that implements a small, fast, self-contained, high-reliability, full-featured, SQL database engine. SQLite is the most used database engine in the world. SQLite is built into all mobile phones and most computers and comes bundled inside countless other applications that people use every day. More Information.

The SQLite file format is stable, cross-platform, and backwards compatible and the developers pledge to keep it that way through the year 2050. SQLite database files are commonly used as containers to transfer rich content between systems and as a long-term archival format for data. There are over 1 trillion SQLite databases in active use.

SQLite source code is in the public-domain and is free to everyone to use for any purpose.



1.2. Front-end

HTML stands for **HyperText Markup Language**. It is the most basic building block of the Web, used to add meaning and structure to web content. Other technologies besides HTML are generally used to describe a web page's appearance/presentation (**CSS**) or functionality/behavior (**JavaScript**).

- **"Hypertext"** refers to the links of various web pages to each other, from one website to another, and vice versa. Links are among the most valuable features of the Web. In a way, by putting your material on the Internet and linking it with pages that other people have created, you are joining and contributing to the World Wide Web.
- **CSS** is a declarative language that controls the styling of web pages in the browser. The browser displays web pages using the CSS style declarations applied to the selected elements. A style declaration consists of properties and their values, which describe the look of a webpage.
- **JavaScript** or "JS," is a high-level, dynamic language used most often for client-side scripting on webpages but is also frequently used on the server-side. Primarily, it is used in the browser to allow developers to change what is going on with the webpage.



1.3. ReportLab

ReportLab is a software library that lets you generate documents directly in Adobe's Portable Document Format (PDF) using the Python programming language. It also generates charts and data graphics in various bitmap and vector formats as well as PDF.

The ReportLab library creates PDF directly from your graphics commands. There are no intervening

steps. Your applications can generate reports extremely fast - sometimes orders of magnitude faster than traditional report-writing tools. This approach is shared by several other libraries - PDFlib for C, iText for Java,

iTextSharp for .NET and others. However, The ReportLab library differs in that it can work at much higher

levels, with a full featured engine for laying out documents complete with tables and charts.



2. Interfaces :

2.1. Page: Login

2.1.1. Purpose:

This page serves as the gateway to the application, enabling users to securely log in to their accounts.

2.1.2. Layout and Features:

The login page is split into two distinct sections, balancing functionality with visual appeal:

Left Section: Login Form

Contains the ParkIT logo at the top, adding branding to the page.

Features a clean and simple login form with:

Email Address Field: Users input their registered email address here.

Password Field: A secure field for entering the account password.

Sign In Button: A black and blue gradient button labeled "Se connecter" submits the login details.

Below the form, a line of text reads, "Vous n'avez pas de compte ?," followed by a "Créer un compte" button. This encourages new users to sign up if they don't already have an account.

Right Section: Welcome Message

This section introduces ParkIT with a brief description of its purpose and features:

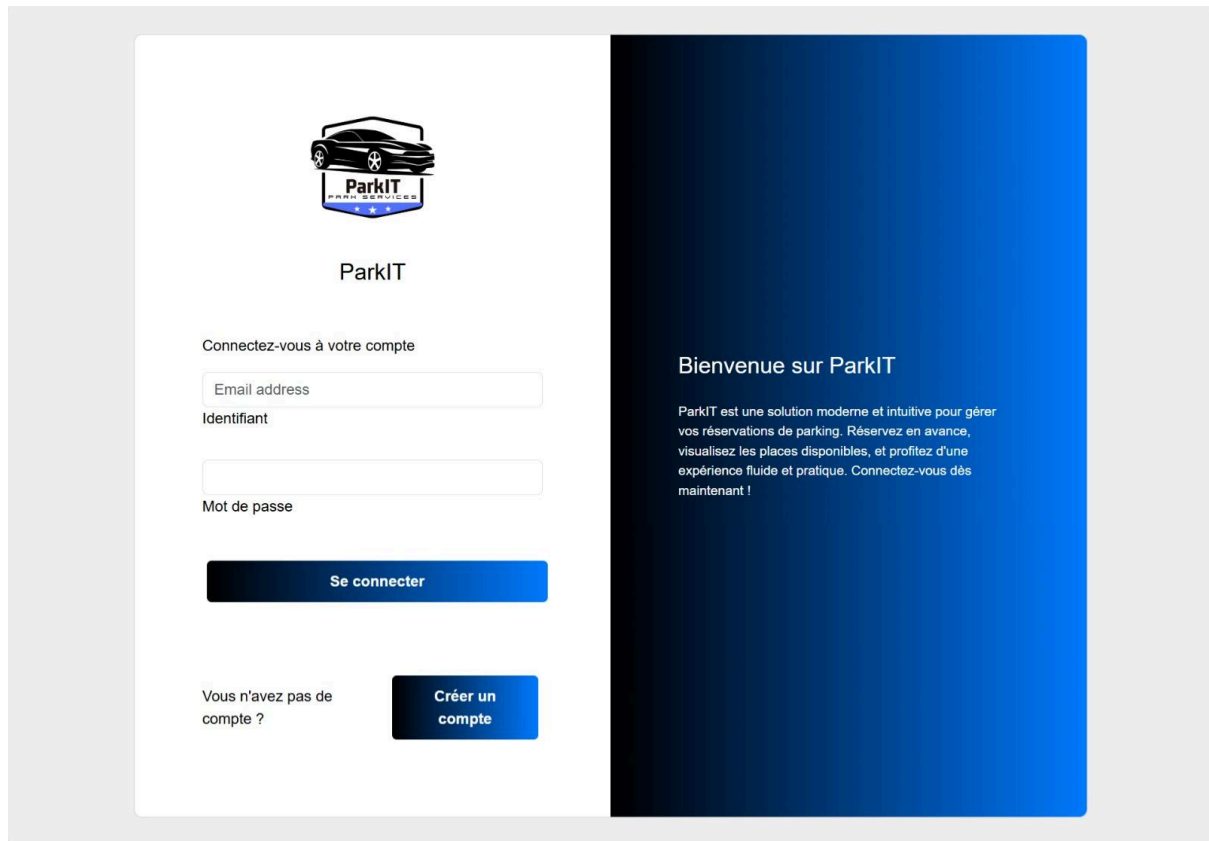
"ParkIT is a modern and intuitive solution for managing your parking reservations. Reserve spots in advance, view available slots, and enjoy a smooth and practical experience."

The gradient background, transitioning from dark blue to light blue, adds a modern and stylish touch.

Visual Design:

The left side features a clean white background, focusing the user's attention on the login form.

The right side uses a gradient background to create visual contrast and a professional appearance.



2.2. Page: Reserve a Parking Spot

2.2.1. Purpose:

This page allows users to reserve a parking spot by providing essential details about their vehicle and the desired reservation times.

2.2.2. Layout and Features:

The page has a clean and minimalist design with a centered form for ease of use. Here's a detailed breakdown of the interface:

License Plate (Matricule):

A text input field where users can enter their vehicle's license plate (e.g., "6H89GK"). This field ensures the reservation is linked to the correct vehicle.

Type of Zone:

A dropdown menu where users can select the type of parking they require. The options include:

- Standard: For regular parking.
- Handicap: For users with accessibility needs.
- VIP: For premium parking zones.

Once a type is selected, the Zone dropdown dynamically updates to show available zones of the chosen type.

Zone:

Another dropdown menu listing the specific parking zones (e.g., "C3") that are currently available.

The list is dynamically populated based on the type of zone selected above, ensuring users see only relevant options.

Start Time and End Time:

Two datetime pickers let users specify the start and end of their reservation. These ensure precise booking details.

Reserve Button:

A prominent blue button labeled "Réserver" submits the form. Upon submission, the reservation details are processed, and the user is redirected to the My Reservations page. Return to Available Spots Button:

A secondary button at the bottom labeled "Retour aux places disponibles" redirects users to the page displaying all available parking slots.

Visual Design:

The form is surrounded by a light gray background with smooth edges, creating a polished look.

The buttons are highlighted in blue, drawing attention to the main actions users can take.

Réserver une place

Matricule :

Type de zone :

Choisissez un type de zone

Zone :

Choisissez une zone

Heure et date de début :

jj/mm/aaaa --:--

Heure et date de fin :

jj/mm/aaaa --:--

Réserver

Retour aux places disponibles

2.3. Page: My Reservations

2.3.1. Purpose:

This page provides users with an overview of their current parking reservations, allowing them to manage their bookings.

2.3.2. Layout and Features:

The page is structured with a header, main content area, and a footer, offering clarity and ease of navigation:

Header:

A bold title, "Mes Réservations," is displayed at the top to clearly indicate the page's purpose.

Reservation Table:

The main content area contains a table displaying all active reservations. The table is divided into the following columns:

- **ID (Réservation):** Displays the unique reservation number for easy identification.
- **Matricule:** Shows the license plate number associated with the reservation (e.g., "6H89GK").
- **Zone:** Indicates the parking zone assigned to the reservation (e.g., "C3").
- **Start and End Times:** Lists the start and end of the reservation in ISO format for clarity and accuracy.
- **Action:** Includes a blue "Annuler" button, allowing users to cancel the reservation. Clicking this button immediately releases the reserved parking spot.
- **Ticket:** Features a blue "Télécharger" button, enabling users to download their reservation ticket as a PDF.

Footer:

At the bottom, a large blue button labeled "Voir les places disponibles" redirects users to the page displaying available parking slots.

Visual Design:

The table uses alternating white and light gray backgrounds for rows to enhance readability. Blue accents highlight actionable elements such as buttons, making them stand out.

Mes Réservations						
ID (RÉSERVATION)	MATRICULE	ZONE	DÉBUT	FIN	ACTION	TICKET
3	6H89GK	C3	2025-01-13T22:13	2025-01-14T22:13	Annuler	Télécharger

Voir les places disponibles

3. Test :

3.1. Unit testing

```
C:\Users\User\Documents\parking_management>py app.py
Traceback (most recent call last):
  File "C:\Users\User\Documents\parking_management\app.py", line 13, in <module>
    init_db()
  File "C:\Users\User\Documents\parking_management\database.py", line 58, in init_db
    cursor.executemany(
      "INSERT INTO parkings (parking_id, zone, type_zone, is_occupied) VALUES (?, ?, ?, ?)",
      parking_slots
    )
sqlite3.IntegrityError: UNIQUE constraint failed: parkings.zone
C:\Users\User\Documents\parking_management>
```

The SQLite IntegrityError screenshot captures a critical issue encountered during the initialization of the database. It highlights a UNIQUE constraint failure for the zone field in the parkings table. This screenshot is relevant for debugging purposes and provides insight into backend constraints that need adjustment. It ensures the developer reviews and resolves database constraints during unit testing.

3.2. Integration test

The "Réserver une place" demonstrates the process where a user enters their license plate (Matricule), selects the type of parking zone (Type), chooses a specific zone (Zone), and provides the start and end date/time for the reservation. This step ensures the system integrates backend logic, database connectivity, and form validation successfully. It validates that the reserve_slot function is working correctly and updates the database.

Réserver une place

Matricule :

6H89GK

Type :

Standard

Zone :

C3

Heure et date de début :

13/01/2025 22:13

Heure et date de fin :

14/01/2025 22:13

Réserver

Retour aux places disponibles

The corresponding "Mes Réservations" screenshot shows the result of the reservation. It confirms that the data is accurately stored in the database and retrieved for display. The integration of the `get_user_reservations` function with the database and frontend is validated here. It also verifies the ability to cancel a reservation or download a ticket, which further demonstrates system interactivity.

Mes Réservations

ID (RÉSERVATION)	MATRICULE	ZONE	DÉBUT	FIN	ACTION	TICKET
3	6H89GK	C3	2025-01-13T22:13	2025-01-14T22:13	Annuler	Télécharger

Voir les places disponibles

3.3. Validation test

The "Mes Réservations" screenshot validates the system's ability to meet user requirements. After completing the reservation process, the user can view their reservations along with the details such as Matricule, Zone, Début, and Fin. The system also provides actionable buttons to cancel the reservation or download the associated ticket. This ensures the user journey is seamless and meets the application's functional goals.

Conclusion générale

Efficient parking management has become an absolute necessity in increasingly crowded and congested cities. In the end, it is all about increased safety and security of citizens, reduced stress, and an improved environment-all factors combined contribute to an improved quality of life. And it's a goal well within reach, with many cities already having much of the infrastructure and technology in place to take advantage.

Bibliographie

- <https://www.python.org/doc/essays/blurb/>
- <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/>
- <https://newsroom.axis.com/blog/parking-management-livable-city>
- https://www.researchgate.net/publication/369466572_Treatment_Strategy_for_Parking_Problems_in_Large_Cities
- <https://www.sqlite.org/>
- <https://en.wikipedia.org/wiki/HTML>
- <https://developer.mozilla.org/en-US/>
- <https://www.reportlab.com/docs/reportlab-userguide.pdf>